

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 03 – KONSEP DASAR



NAMA: Nur Tsalits Alwan Mubarok

NIM: 103132400039

KELAS: S1SD-05-01

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

1) Pemrograman Berorientasi Objek (PBO)

PBO adalah paradigma pemrograman yang membangun sistem dari kumpulan objek. Setiap objek membungkus data dan fungsi dalam satu kesatuan, lalu objek-objek tersebut saling berinteraksi. Karakteristik utamanya adalah abstraksi, enkapsulasi, pewarisan, dan polimorfisme.

2) Abstraksi

Abstraksi adalah proses mengambil hal-hal yang esensial dari suatu objek dan mengabaikan detail yang tidak penting. Hasilnya berupa atribut (ciri-ciri, berupa kata benda atau sifat) dan perilaku (hal yang dapat dilakukan, berupa kata kerja)

3) Enkapsulasi

Enkapsulasi adalah pemaketan data dan method dalam satu class, sambil menyembunyikan rincian internalnya dari luar. Data dibuat private dan hanya diakses melalui method public. Dengan begitu, data terlindungi dari perubahan langsung dan bagian internal class bisa diubah tanpa memengaruhi kode lain.

4) Class dan Object

Class adalah cetak biru yang mendefinisikan atribut dan method. Object adalah hasil instansiasi dari class dan dibuat dengan kata kunci new, misalnya `Kucing aren = new Kucing();`. Dari satu class dapat dibuat banyak object.

5) Field: Instance Field dan Class Field

Field adalah atribut yang menyimpan data. Instance field dideklarasikan tanpa static, dan setiap object memiliki salinannya sendiri. Class field dideklarasikan dengan static, dimiliki oleh class, dan dipakai bersama oleh semua object. Jika ditambah final, nilainya tetap dan tidak dapat diubah (konstanta).

6) Method dan Constructor

Method adalah perilaku dari sebuah class. Method yang mengembalikan nilai disebut fungsi (memakai return), sedangkan yang tidak mengembalikan nilai disebut prosedur (void). Constructor adalah method khusus yang bernama sama dengan class dan dijalankan otomatis saat object dibuat, biasanya untuk mengisi nilai awal field.

7) Keyword this

this adalah referensi ke object yang sedang aktif. Keyword ini dipakai untuk membedakan field dari parameter ketika namanya sama, misalnya `this.nama = nama;`.

8) Pass by Value dan Pass by Reference

Pada tipe data primitif (int, double, dan sebagainya), yang dikirim ke method hanyalah salinan nilai, sehingga perubahan di dalam method tidak memengaruhi variabel asal. Pada array dan object, yang dikirim adalah referensi (alamat memori), sehingga perubahan di dalam method ikut terlihat di variabel asal.

B. Guided

1. Abstraksi

```
//file kucing.java
public class Kucing {
    String nama;
    String ras;

    void bersuara(){
        System.out.println(nama + ras + "meow");
    }
}

//file abstraksi.java
public class Abstraksi {
    public static void main(String[] args){
        Kucing aren = new Kucing();

        aren.nama = "Aren Gentong";
        aren.ras = "Mujaer";
        aren.bersuara();
    }
}
```

Output :

```
Aren GentongMujaermeow
-----
```

2. Enkapsulasi

```
//file Rekening.java
public class Rekening {
    private int saldo = 0;

    public void tambahSaldo(int jumlah){
        saldo = saldo + jumlah;
        System.out.println("Saldo berhasil ditambahkan");
    }

    public void tampilkanSaldo(){
        System.out.println("Saldo Anda: " + saldo);
    }
}
```

```
//file Enkapsulasi.java
public class Enkapsulasi {
    public static void main(String[] args){
        Rekening rek = new Rekening();

        rek.tambahSaldo(20000000);
        rek.tampilkanSaldo();
    }
}
```

Output :

```
Saldo berhasil ditambahkan
Saldo Anda: 20000000
-----
```

C. Unguided

1. Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah. Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya. Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Kebutuhan program

Class ini harus dapat:

1. ****Menyimpan data suhu 7 hari****, dibuat dari array suhu mentah yang sudah ada di `main` (index array mewakili hari).
2. ****Menyimpan nilai penanda data kosong (`-1.0`)** sebagai aturan tetap milik class**, yaitu class field yang nilainya tidak dapat diubah dan dipakai bersama oleh semua object.
3. ****Menampilkan seluruh data apa adanya****, menandai hari yang kosong secara jelas.
4. ****Mencari apakah ada hari yang kosong, dan di index berapa****, karena index inilah yang nanti dipakai untuk proses pengisian data.
5. ****Mengisi hari yang kosong****, menggunakan index dari poin 4, dengan nilai perkiraan berdasarkan rata-rata suhu satu hari sebelum dan satu hari sesudahnya.
6. ****Menghitung rata-rata suhu**** dari data yang sudah bersih.

Jawaban:

```
import java.util.Locale;
import java.io.PrintStream;
import java.nio.charset.StandardCharsets;

class PengolahSuhu {

    // Class field: nilai penanda data kosong. static = milik class (dipakai bersama
    // semua object), final = nilainya tidak dapat diubah. Ditulis satu kali saja.
    private static final double NILAI_KOSONG = -1.0;

    // Penanda "tidak ditemukan" untuk hasil pencarian index
    private static final int TIDAK_DITEMUKAN = -1;

    // Instance field: berbeda untuk setiap object. private agar tidak bisa
    // diakses/diubah langsung dari luar class (enkapsulasi).
    private double[] suhuHarian;

    // Constructor: nama parameter sama dengan nama field, jadi pakai this.
    public PengolahSuhu(double[] suhuHarian) {
        this.suhuHarian = suhuHarian; // menyimpan REFERENSI, bukan menyalin
        array
    }

    // Kebutuhan 3: tampilkan seluruh data, hari kosong ditandai "(kosong)"
    public void tampilkanData() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                System.out.println("Hari " + (i + 1) + " : (kosong)");
            } else {
                System.out.println(String.format(Locale.US, "Hari %d : %.1f°C", i + 1,
suhuHarian[i]));
            }
        }
    }

    // Kebutuhan 4: cari index hari kosong, return TIDAK_DITEMUKAN (-1) jika
    tidak ada
    public int cariIndexKosong() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                return i;
            }
        }
    }
}
```

```

        return TIDAK_DITEMUKAN;
    }

    // Kebutuhan 5: isi data kosong dengan rata-rata hari sebelum dan sesudahnya
    public void isiDataKosong() {
        int i = cariIndexKosong();
        if (i != TIDAK_DITEMUKAN) {
            suhuHarian[i] = (suhuHarian[i - 1] + suhuHarian[i + 1]) / 2;
        }
    }

    // Kebutuhan 6: rata-rata suhu dari data yang sudah bersih
    public double hitungRataRata() {
        double total = 0;
        for (double suhu : suhuHarian) {
            total += suhu;
        }
        return total / suhuHarian.length;
    }
}

public class Main {
    public static void main(String[] args) {
        System.setOut(new PrintStream(System.out, true,
StandardCharsets.UTF_8));
        // 1. Siapkan array
        double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };

        // 2. Buat object, tampilkan data awal, cari index kosong
        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();
        System.out.println();
        System.out.println("Index hari kosong (dimulai dari 0): " +
pengolah.cariIndexKosong());
        System.out.println();

        // 3. Isi data kosong
        pengolah.isiDataKosong();

        // 4. Tampilkan data akhir + rata-rata
        System.out.println("=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();
    }
}

```

```

        System.out.println();
        System.out.println(String.format(Locale.US, "Rata-rata : %.2f°C",
pengolah.hitungRataRata()));
        System.out.println();

        // 5. Tampilkan array suhuHarian milik main (bukan lewat object)
        System.out.println("Isi array suhuHarian di main setelah isiDataKosong()
dijalankan:");
        StringBuilder sb = new StringBuilder("");
        for (int i = 0; i < suhuHarian.length; i++) {
            sb.append(String.format(Locale.US, "%.1f", suhuHarian[i]));
            if (i < suhuHarian.length - 1) {
                sb.append(", ");
            }
        }
        sb.append("]");
        System.out.println(sb);
        System.out.println("(ikut berubah: constructor menyimpan referensi array
yang sama)");

```

```

        // PENJELASAN: Array di Java bertipe reference. Variabel suhuHarian di
main
        // tidak menyimpan isi array, melainkan alamat (referensi) ke array di
memori.
        // Saat constructor PengolahSuhu dipanggil, yang disalin ke field
        // this.suhuHarian hanyalah alamat tersebut, bukan isi array-nya. Akibatnya,
        // variabel di main dan field di dalam object menunjuk ke SATU array yang
sama
        // persis. Ketika isiDataKosong() mengubah suhuHarian[3] lewat object,
array
        // yang diubah adalah array yang sama dengan milik main, sehingga
perubahan
        // itu otomatis terlihat dari main walaupun main tidak mengubahnya secara
        // langsung. (Agar tidak ikut berubah, constructor harus membuat salinan,
        // misalnya this.suhuHarian = suhuHarian.clone();)
    }

```

```

}

```

Output:

```
=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
(ikut berubah: constructor menyimpan referensi array yang sama)
```

Penjelasan:

Program ini terdiri dari dua class, yaitu PengolahSuhu untuk mengolah data suhu dan Main untuk menjalankan program.

1. Class PengolahSuhu

- Class field NILAI_KOSONG: dideklarasikan private static final double dengan nilai -1.0. Karena static, field ini milik class dan dipakai bersama oleh semua object. Karena final, nilainya tidak dapat diubah. Nilai penanda cukup ditulis sekali dan dipakai di semua method.
- Instance field suhuHarian: bertipe double[] dan dideklarasikan private sehingga tidak dapat diakses langsung dari luar class (enkapsulasi). Isinya berbeda untuk setiap object.
- Constructor: menerima array suhu lalu menyimpannya ke field. Nama parameter sama dengan nama field, sehingga dipakai this.suhuHarian = suhuHarian untuk membedakan field dari parameter.
- tampilkanData(): menampilkan suhu tiap hari dan menandai hari yang bernilai NILAI_KOSONG dengan tulisan (kosong).
- cariIndexKosong(): menelusuri array dan mengembalikan index hari yang kosong, atau -1 jika tidak ada.
- isiDataKosong(): memanggil cariIndexKosong(). Jika hasilnya bukan -1, data di index tersebut diisi dengan rata-rata suhu hari sebelum dan sesudahnya, yaitu $(\text{suhuHarian}[i-1] + \text{suhuHarian}[i+1]) / 2$.
- hitungRataRata(): menjumlahkan seluruh suhu lalu membaginya dengan jumlah hari.

2. Class Main

Method main menyiapkan array suhu, membuat object PengolahSuhu, menampilkan data awal beserta index data kosong, mengisi data kosong, lalu menampilkan data akhir dan rata-rata suhu. Terakhir, array suhuHarian di main ditampilkan lagi.

3. Hasil dan Analisis

Hari 4 yang semula kosong terisi 29.1°C (hasil dari $(26.8 + 31.4) / 2$), dan rata-rata suhu adalah 29.39°C . Array suhuHarian di main ikut berubah walaupun tidak diubah langsung. Ini karena array bertipe reference: constructor hanya menyimpan alamat array, bukan menyalinnya, sehingga variabel di main dan field di dalam object menunjuk ke array yang sama.

D. Kesimpulan

Berdasarkan praktikum Modul 2 ini, kesimpulan nya bahwa:

Dari praktikum Modul 3 dapat disimpulkan bahwa pemrograman berorientasi objek membangun program dari objek-objek yang dibuat berdasarkan class. Class berfungsi sebagai cetak biru yang berisi atribut (field) dan perilaku (method), sedangkan object adalah hasil instansiasinya. Konsep abstraksi diterapkan dengan mengambil hal-hal yang esensial dari suatu objek, seperti nama, ras, dan bersuara() pada class Kucing. Konsep enkapsulasi diterapkan dengan menyembunyikan data menggunakan modifier private sehingga data tidak dapat diubah langsung dari luar class.

Pada soal unguided, instance field suhuHarian menyimpan data yang berbeda untuk setiap object, sedangkan class field NILAI_KOSONG (static final) menyimpan nilai tetap yang dipakai bersama oleh semua object. Keyword this dipakai pada constructor untuk membedakan field dari parameter yang bernama sama. Hasil praktikum juga menunjukkan bahwa array bertipe reference: constructor hanya menyimpan alamat array, sehingga perubahan yang dilakukan lewat object ikut terlihat pada array asal di main.

Hari 4 yang semula kosong terisi 29.1°C dan rata-rata suhu yang dihasilkan adalah 29.39°C

